

Cloud Computing

(W4153_2025_03)

Lecture 2: IaaS, Microservices, REST



Contents

- Infrastructure-as-a-Service
 - Concepts
 - Virtualization
 - Virtual Private Cloud
 - Cloud virtual machines
 - Deploy first virtual machine
- Microservices
 - Concepts and characteristics
 - Architecture best practices: 12-Factor Apps, SOLID
 - Design patterns
- REST
 - Concepts
 - A basic implementation example
- API First and OpenAPI
- Agile Development
- Sprint 1 Team Project

Infrastructure-as-a-Service

Infrastructure-as-a-Service

Infrastructure-as-a-Service

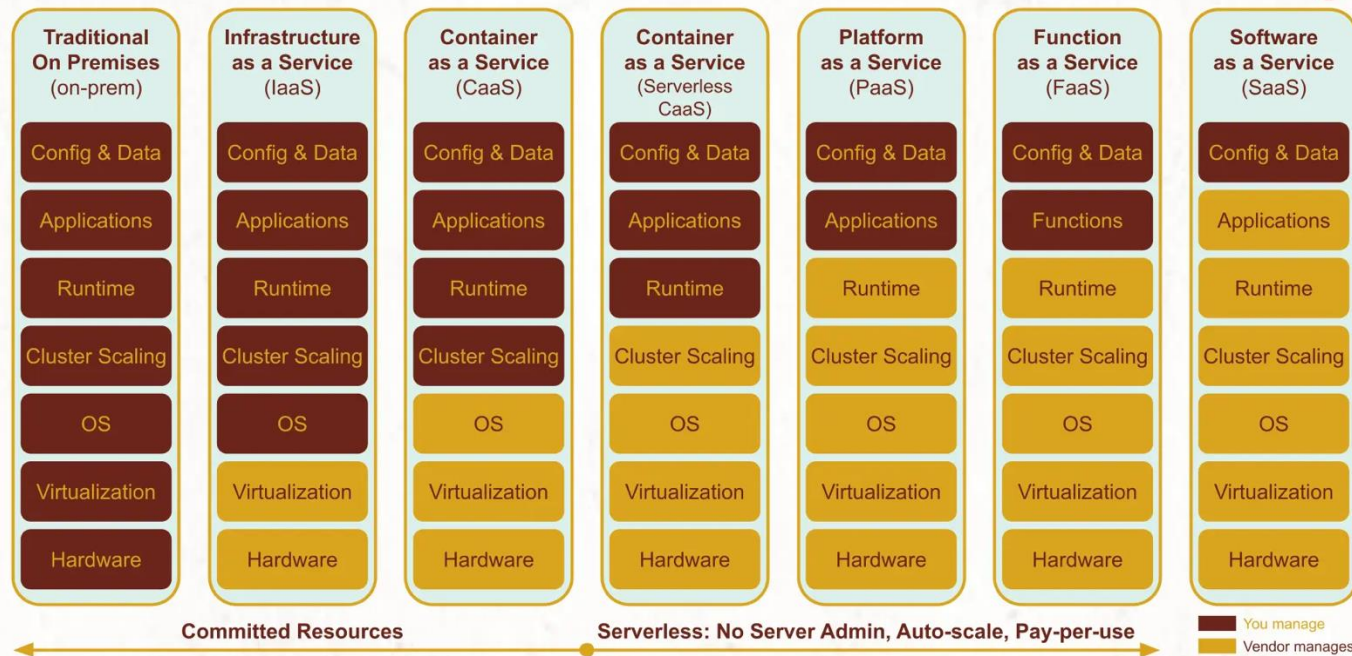
“Infrastructure as a service (IaaS) is a cloud computing service model where a cloud services vendor provides computing resources such as storage, network, servers, and virtualization (which emulates computer hardware). This service frees users from maintaining their own data center,[1] but they must install and maintain the operating system and application software. [2] IaaS provides users high-level APIs to control details of underlying network infrastructure such as backup, data partitioning, scaling, security and physical computing resources. Services can be scaled on-demand by the user. According to the Internet Engineering Task Force (IETF), such infrastructure is the most basic cloud-service model.”

https://en.wikipedia.org/wiki/Infrastructure_as_a_service

Cloud Layers

Cloud Deployment Spectrum

ml4devs.com/serverless 



 © Satish Chandra Gupta, Creative Commons BY-NC-ND 4.0 International License.

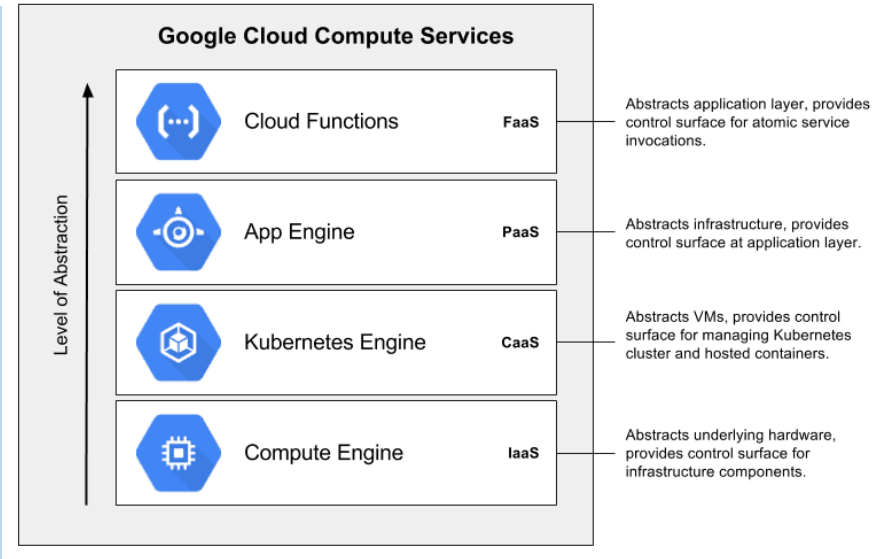
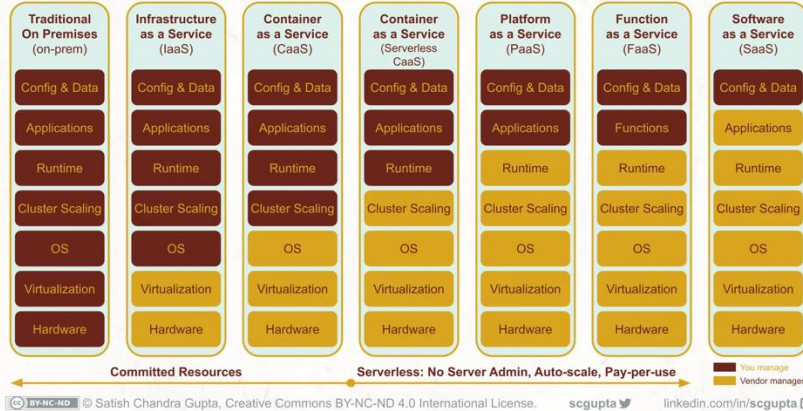
scgupta 

linkedin.com/in/scgupta 

Cloud Layers

Cloud Deployment Spectrum

ml4devs.com/serverless

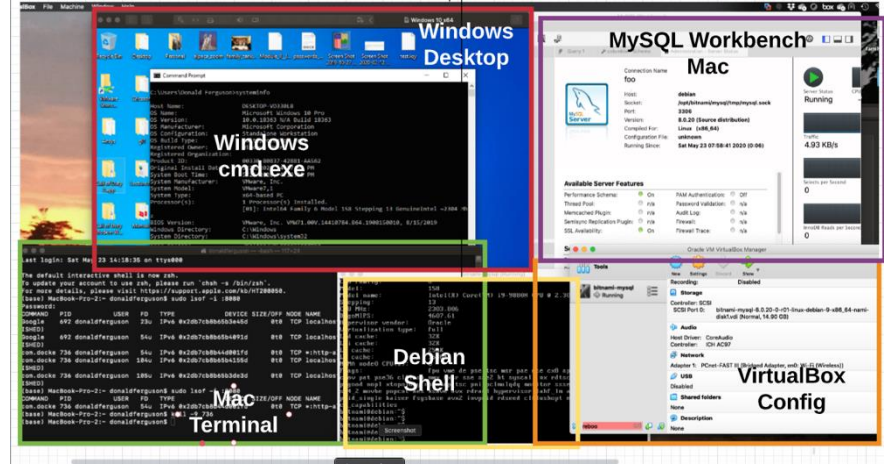
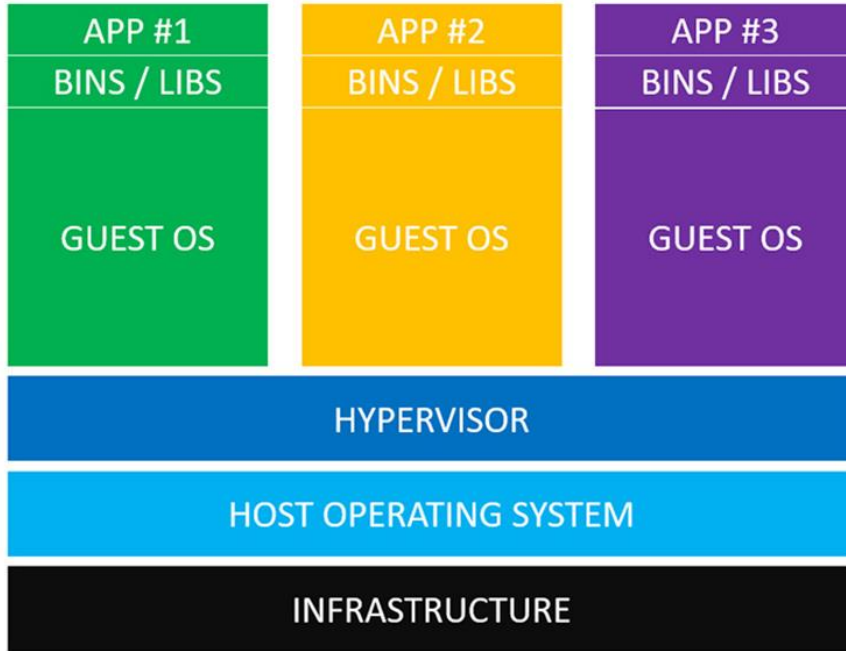


- To understand the concepts, we will “work our way up the stack.”
- This will help us understand the complexity, pros and cons.
- This means we start with *Google Cloud Platform Compute Engine*. – The basic building blocks is a “virtual machine.”

Virtualization

Virtualization, Virtual Machines

- "In computing, a virtual machine (VM) is an emulation of a computer system. Virtual machines are based on computer architectures and provide functionality of a physical computer. Their implementations may involve specialized hardware, software, or a combination." (https://en.wikipedia.org/wiki/Virtual_machine)



On my Mac laptop, I used to have:

- One host operating system and HW (Mac, MacOS)
- Two hypervisors (Virtual Box, VMWare Fusion)
- Two guest OS: Windows 10, Debian Linux

Then, ...

- Apple transitioned to ARM
- Broadcom bought VMWare

Then people started asking, "Why do you want to do this grandpa?"

Hypervisor

- “A hypervisor, also known as a virtual machine monitor (VMM) or virtualizer, is a type of computer software, firmware or hardware that creates and runs virtual machines. A computer on which a hypervisor runs one or more virtual machines is called a host machine, and each virtual machine is called a guest machine. The hypervisor presents the guest operating systems with a virtual operating platform and manages the execution of the guest operating systems.” (<https://en.wikipedia.org/wiki/Hypervisor>)
- Instructor's Note:**
 - If I let you use Google, Stack Overflow, GitHub, to write code, solve problems, ...
 - I get to use Google, Stack Overview, GitHub, to make my lecture slides.



What are the benefits of a hypervisor?

Organizations use virtualization software like hypervisors because the software helps them to use resources efficiently and reduce hardware investment. Virtualization brings several other benefits such as those given below.

Hardware independence

A hypervisor abstracts the host's hardware from the operating software environment. IT administrators can configure, deploy, and manage software applications without being constrained to a specific hardware setup. For example, you can run macOS on a virtual machine instead of iMac computers.

Efficiency

Hypervisors make setting up a server operating system more efficient. Manually installing the operating system and related software components is a time-consuming process. Instead, you can configure the hypervisor to immediately create your virtual environment.

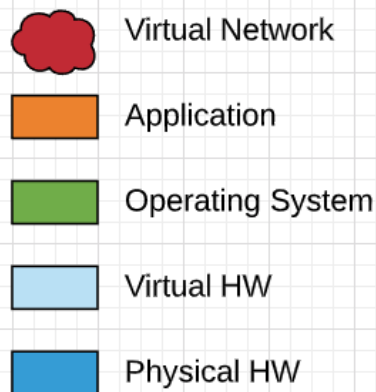
Scalability

Organizations use hypervisors to maximize resource usage on physical computers. Instead of using separate machines for different workloads, hypervisors create multiple virtual computers to run several workloads on a single machine. This translates to faster scalability and reduced hardware expenditure for organizations.

Portability

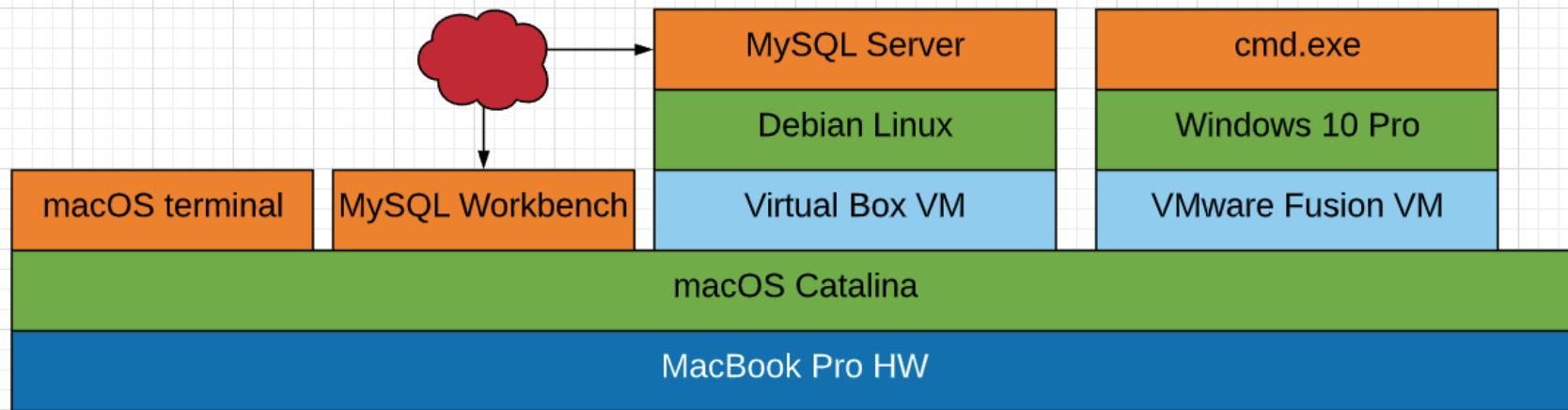
IT teams can allocate memory, networking, processing, and storage resources across multiple servers as needed. They have the ability to shift workloads between machines or platforms easily. When an application requires more processing power, the hypervisor provides seamless access to additional physical resources.

Virtual Machines on Physical Machines

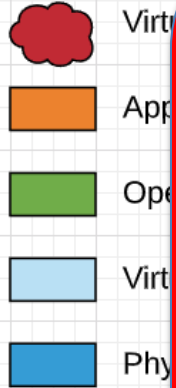


Note:

- I did not replicate this setup on my new Mac.
 - Contrary to what you might think, I do have other work to do.
 - I have Ubuntu on top of UTM.
- But, I cannot bridge the network!



Virtual Machines on Physical Machines



I went down the VMWare Fusion, UTM, Ubuntu, ... Rabbit Hole and wasted hours trying to make this useless example work.

Summary:
IaaS uses VMs.
VMs are cool.

Do not work about VMs. They are magic.

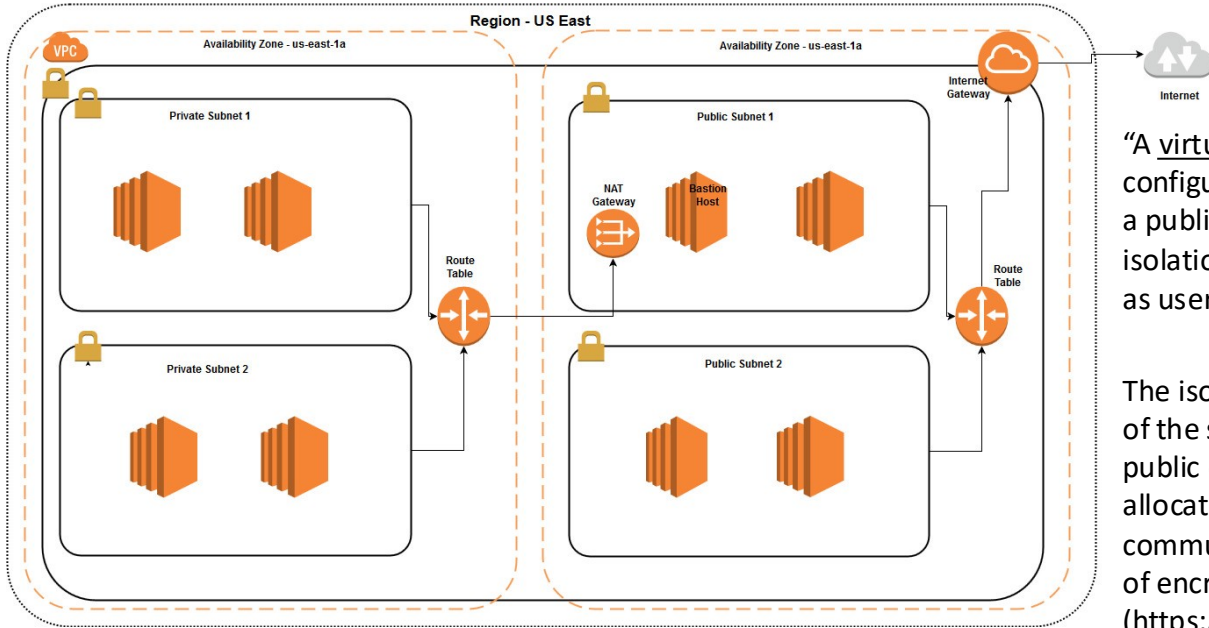
macOS

macOS Catalina

MacBook Pro HW

Virtual Private Cloud and VM

Virtual Private Cloud

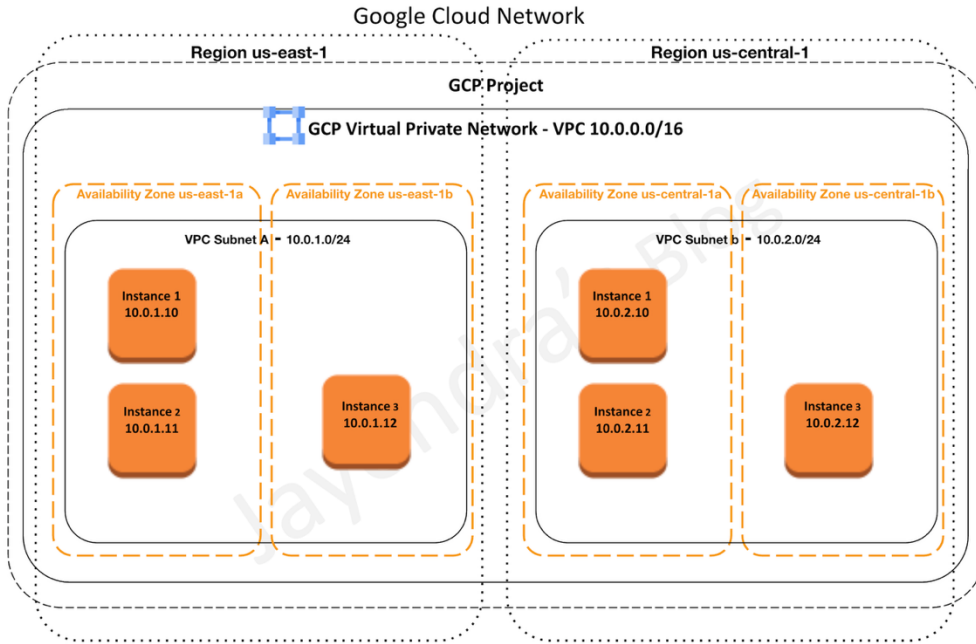


“A virtual private cloud (VPC) is an on-demand configurable pool of shared resources allocated within a public cloud environment, providing a certain level of isolation between the different organizations (denoted as users hereafter) using the resources.

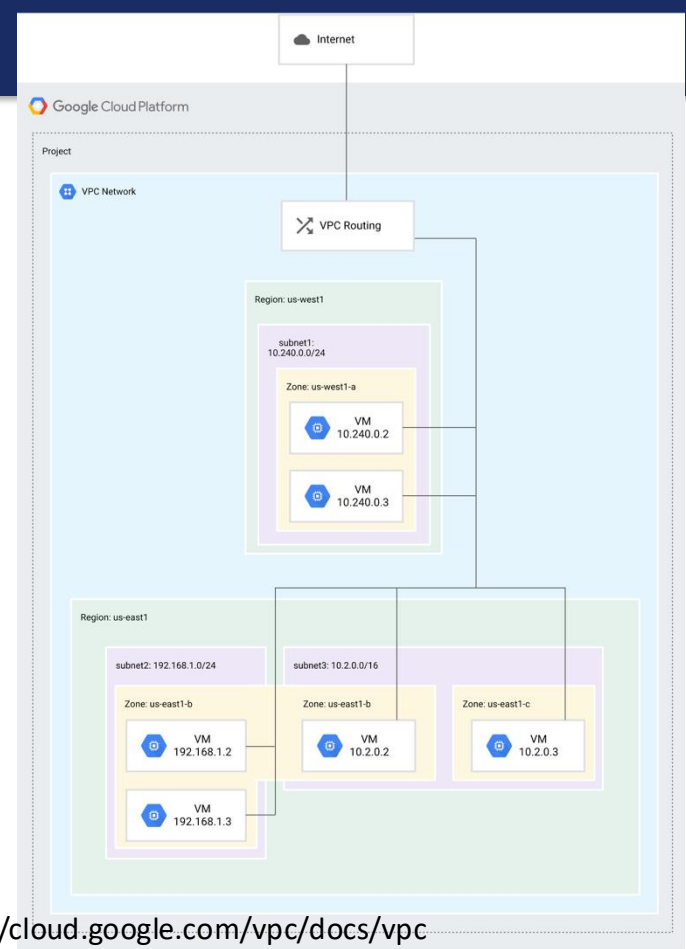
The isolation between one VPC user and all other users of the same cloud (other VPC users as well as other public cloud users) is achieved normally through allocation of a private IP subnet and a virtual communication construct (such as a VLAN or a set of encrypted communication channels) per user.”
(https://en.wikipedia.org/wiki/Virtual_private_cloud)

- A virtual machine on my laptop has a virtual adaptor connected to a real network.
- A virtual private cloud is a set of virtual machines connected to a virtual network.
- All cloud providers support virtual private clouds with the same concepts, but with slightly different realizations and terms.

Google Cloud VPC



<https://jayendrapatil.com/google-cloud-virtual-private-cloud-vpc/>



<https://cloud.google.com/vpc/docs/vpc>

Some Terminology (AWS) – GCP Is Similar

There are several online tutorials. Understanding some terms is useful.

- AWS has the concept of a Region, which is a physical location around the world where we cluster data centers. We call each group of logical data centers an Availability Zone. Each AWS Region consists of multiple, isolated, and physically separate AZ's within a geographic area.
- An Availability Zone (AZ) is one or more discrete data centers with redundant power, networking, and connectivity in an AWS Region. AZs give customers the ability to operate production applications and databases that are more highly available, fault tolerant, and scalable than would be possible from a single data center.
- A VPC is a virtual private cloud, which works like a private network to isolate the resources within it.
- A route table contains a set of rules, called routes, that are used to determine where network traffic is directed.
- A subnet is a defined set of network IP addresses that are used to increase the security and efficiency of network communications. You can think of them like postal codes, used for routing packages from one location to another.
- A network interface represents a virtual network card. The network interface displays its network interface ID, subnet ID, VPC ID, security group, and the Availability Zone that it exists in.
- A security group is a set of rules that controls the network access to the resources it is associated with. Access is permitted only to and from the components defined in the security group's inbound and outbound rules.
- An internet gateway is a VPC component that allows communication between instances in your VPC and the internet.
- A VPC endpoint enables you to privately connect your VPC to supported AWS services without using public IP addresses.

All clouds have similar concepts but with different names.

Let's Build our First VPC and VM

- Well, actually, let's not do that for now.
- We are just going to create our first virtual machines
- But,

- The
- Sett

- We can
- I am going

- I will
- Basic
- 1.
- 2.
- 3.

- There is a

Just show VM on GCP.

Follow the tutorials. It is simple and quick.

<https://github.com/donald-f-ferguson/W4153-Hello-World-FastAPI.git>

What Comes/Would Come Next?

- We did the VM work “manually,” and this was:
 - Tedious
 - Complex
- An organization typically has two “teams:”
 - Develop team
 - Infrastructure team
- The infrastructure team has a “library” of prebuilt/previously built environments. Provides automation tools to simplify iterative development and deployment.
- A development team can:
 - Create a new instance of a predefined environment.
 - Request creation or modification of an environment.
- Terms in this space are continuous integration/continuous delivery and DevOps.
- We do things manually in this course for education/understanding purposes. We will start automating in a couple of lectures.

Microservices

Reminder and Continuation

Reminder

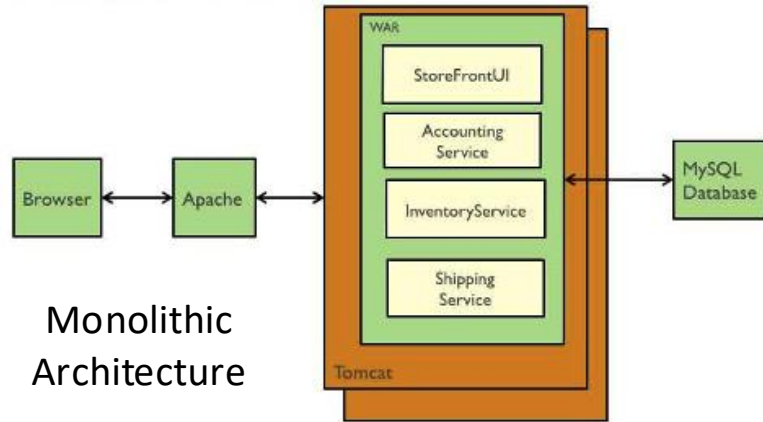
Microservices (<https://microservices.io/index.html>)

What are microservices?

Microservices - also known as the microservice architecture - is an architectural style that structures an application as a collection of services that are

- Highly maintainable and testable
- Loosely coupled
- Independently deployable
- Organized around business capabilities
- Owned by a small team

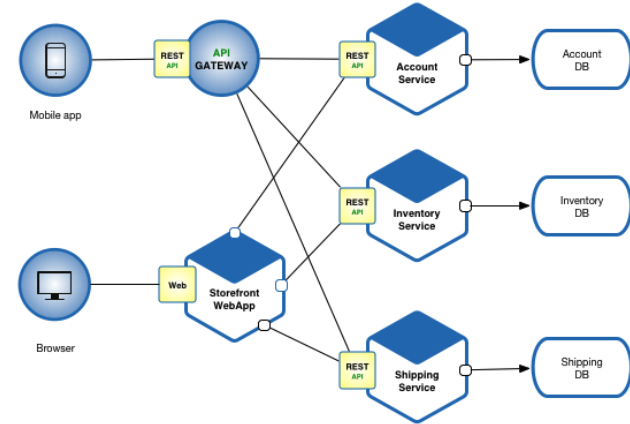
The microservice architecture enables the rapid, frequent and reliable delivery of large, complex applications. It also enables an organization to evolve its technology stack.



Monolithic
Architecture

- Microservice benefits relative to monoliths are hard to understand without having developed and maintained monoliths.
- Cars are “cool.” Cars are totally, super cool if you had to go into town using a horse drawn carriage.

Microservices Architecture





Event based --> Loose coupled

Microservice Characteristics

- **Decoupling** - Services within a system are largely decoupled, so the application as a whole can be easily built, altered, and scaled.
- **Componentization** - Microservices are treated as independent components that can be easily replaced and upgraded.
- **Business Capabilities** - Microservices are very simple and focus on a single capability.
- **Autonomy** - Developers and teams can work independently of each other, thus increasing speed.
- **Continuous Delivery** - Allows frequent releases of software through systematic automation of software creation, testing, and approval.
- **Responsibility** - Microservices do not focus on applications as projects. Instead, they treat applications as products for which they are responsible.
- **Decentralized Governance** - The focus is on using the right tool for the right job. That means there is no standardized pattern or any technology pattern. Developers have the freedom to choose the best useful tools to solve their problems.
- **Agility** - Microservices support agile development. Any new feature can be quickly developed and discarded again.

A student asked something like,
"Is there more to microservices than
just breaking things into parts?"



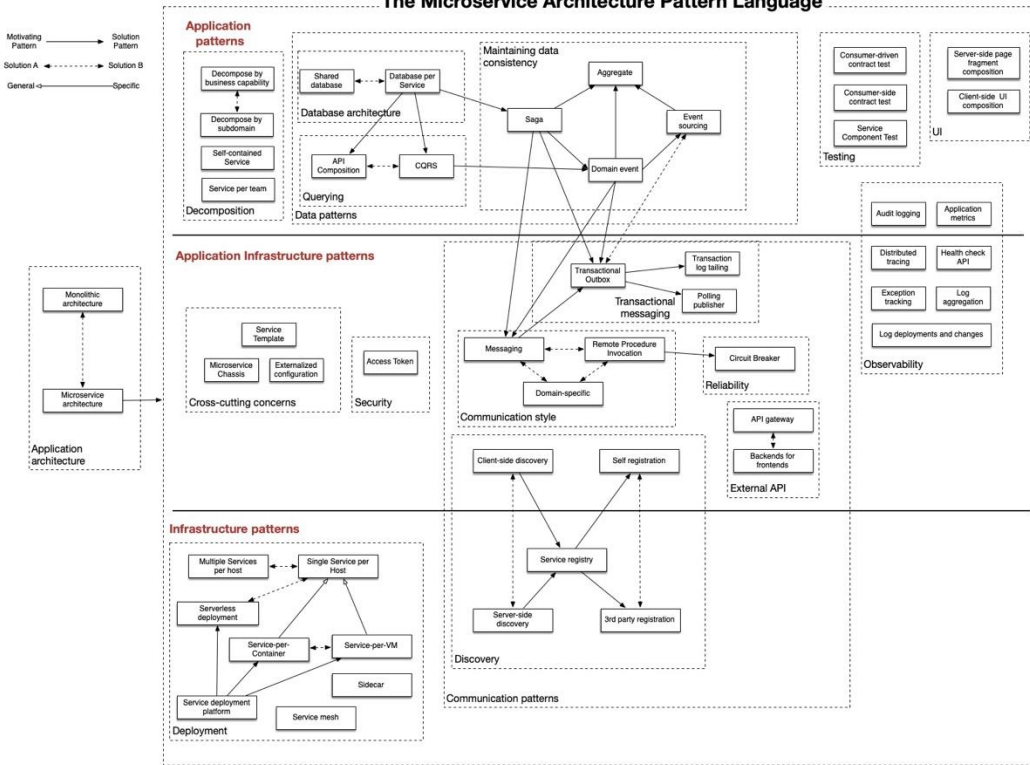
Design patterns and Design Principles

Design Patterns

- “In software engineering, a software design pattern is a general, reusable solution to a commonly occurring problem within a given context in software design. It is not a finished design that can be transformed directly into source or machine code. Rather, it is a description or template for how to solve a problem that can be used in many different situations. Design patterns are formalized best practices that the programmer can use to solve common problems when designing an application or system.”
(https://en.wikipedia.org/wiki/Software_design_pattern)
- We will cover design patterns in several aspects:
 - SW patterns for building code “in microservices.” How do you write the code?
 - Architecture patterns for microservices and composition into complex solutions.
 - Cloud architecture patterns: how do you solve problems with cloud technology.
 - Data modeling and data access patterns.
 -

Microservice Pattern Language (One Perspective)

The Microservice Architecture Pattern Language



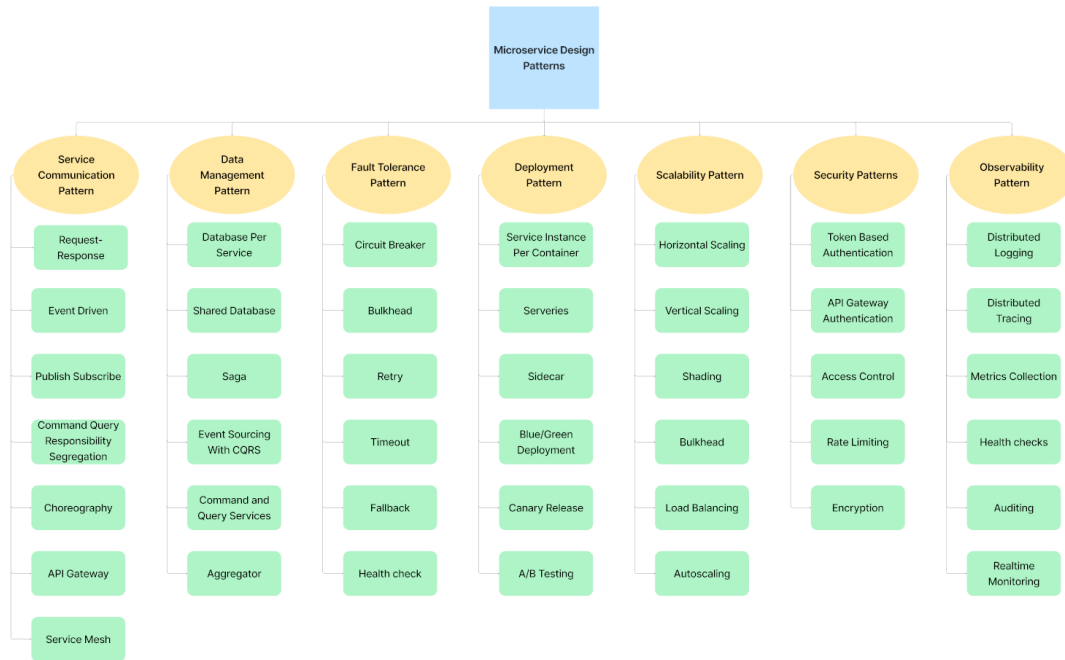
Copyright © 2022, Chris Richardson Consulting, Inc. All rights reserved.

Learn-Build-Assess Microservices <http://adopt.microservices.io>

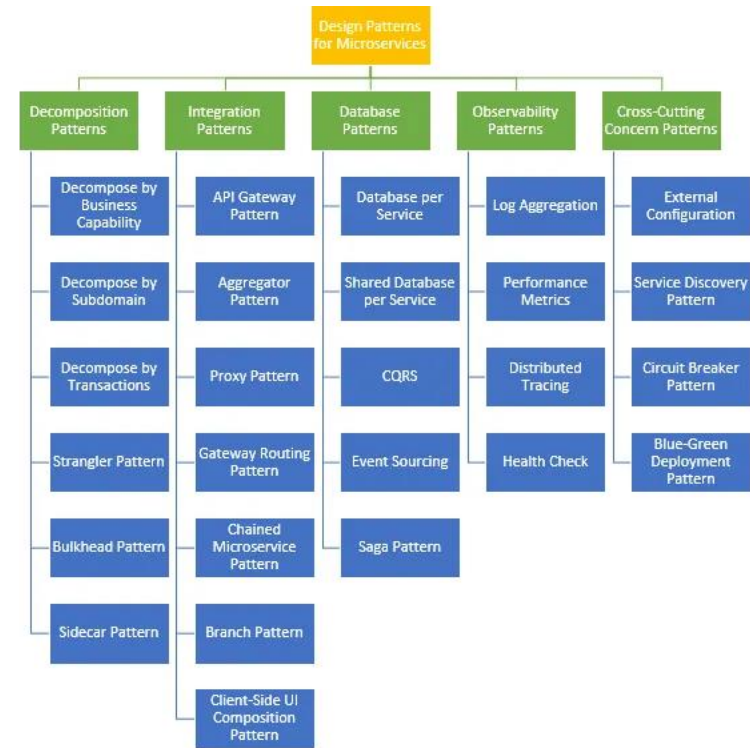
- If you are trying to solve a problem or implement something,
- You are probably not the first person to do so.
- Probably a lot of other people have tried, failed and then learned.
- Ask Google or ChatGPT what others do.
- A knowledge of basic design patterns is useful to guide architecture/implementation decisions.
- We will cover when encountering tasks and challenges.

<https://microservices.io/patterns/index.html>

There are a Lot of Design Patterns



<https://blog.beehyv.com/design-patterns-in-microservices-enhancing-scalability-flexibility-and-maintainability-197e499a575b>



<https://medium.com/@madhukaudantha/microservice-architecture-and-design-patterns-for-microservices-e0e5013fd58a>

12 Factor Applications

<https://dzone.com/articles/12-factor-app-principles-and-cloud-native-microser>



12 Factor App Principles

Codebase

One codebase tracked in revision control, many deploys

Dependencies

Explicitly declare and isolate the dependencies

Config

Store configurations in an environment

Backing Services

Treat backing resources as attached resources

Build, release, and, Run

Strictly separate build and run stages

Processes

Execute the app as one or more stateless processes

Port Binding

Export services via port binding

Concurrency

Scale-out via the process model

Disposability

Maximize the robustness with fast startup and graceful shutdown

Dev/prod parity

Keep development, staging, and production as similar as possible

Logs

Treat logs as event streams

Admin processes

Run admin/management tasks as one-off processes

SOLID Principles

blog.bytebytego.com

S

Single Responsibility Principle (SRP)

A class should have only one reason to change, meaning it should have a single, well-defined responsibility.

O

Open/Closed Principle (OCP)

Software entities (e.g., classes, modules) should be open for extension but closed for modification. This promotes the idea of extending functionality without altering existing code.

L

Liskov Substitution Principle (LSP)

Subtypes (derived classes) must be substitutable for their base types (parent classes) without altering the correctness of the program.

I

Interface Segregation Principle (ISP)

Clients should not be forced to depend on interfaces they don't use. This principle encourages the creation of smaller, focused interfaces.

D

Dependency Inversion Principle (DIP)

High-level modules should not depend on low-level modules; both should depend on abstractions. This promotes the decoupling of components through abstractions and interfaces.

Microservices Guidelines

SOLID Principles

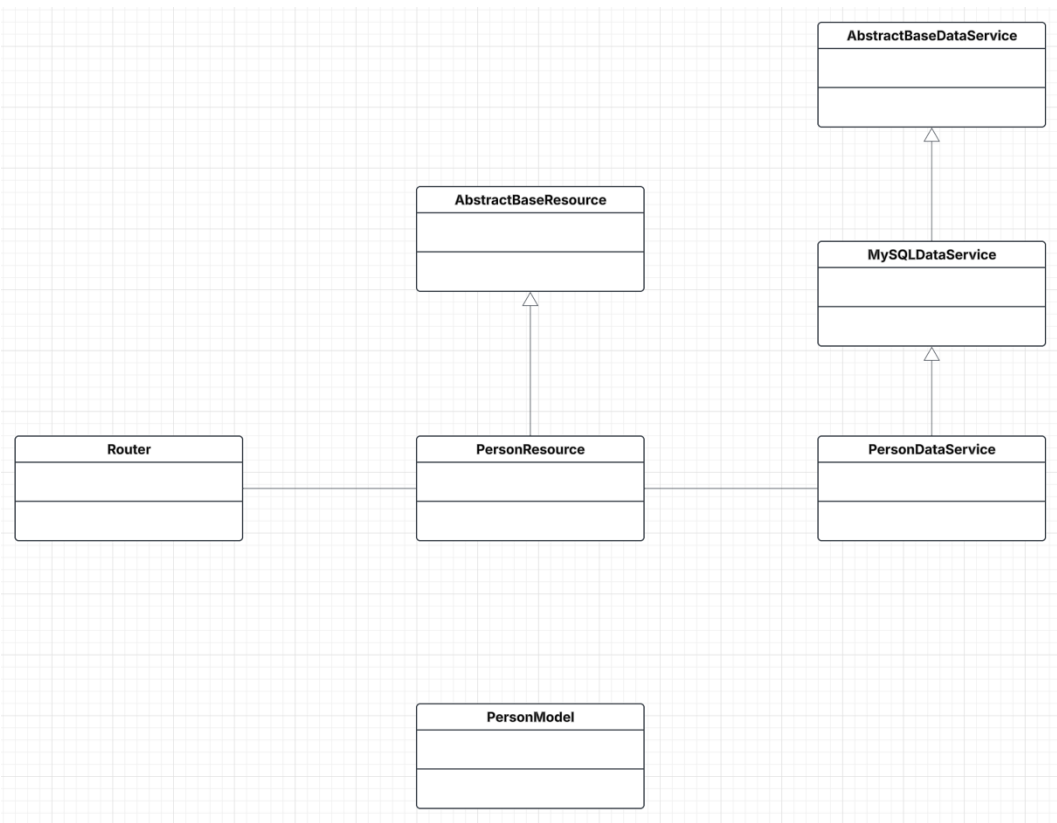
Loosely coupled	Interface Segregation + Dependency Inversion
Testable	Single Responsibility + Dependency Inversion.
Composable	Single Responsibility + Open/close
Highly maintainable	Single Responsibility + Liskov Substitution + Interface Segregation
Independently deployable	Single Responsibility + Interface Segregation + Dependency Inversion
Capable of being developed by a small team	Single Responsibility + Interface Segregation

<https://medium.com/@saurabh.engg.it/microservices-designing-effective-microservices-by-following-solid-design-principles-a995c3a033a0>

This is a good overview article, and we will see patterns in later lectures.

- This slide and the preceding slides could have been a 60 slide presentation.
- If you put 3 SW architects into a room, you will get 15 design principles and patterns.
- Use common sense.

SOLID Principle



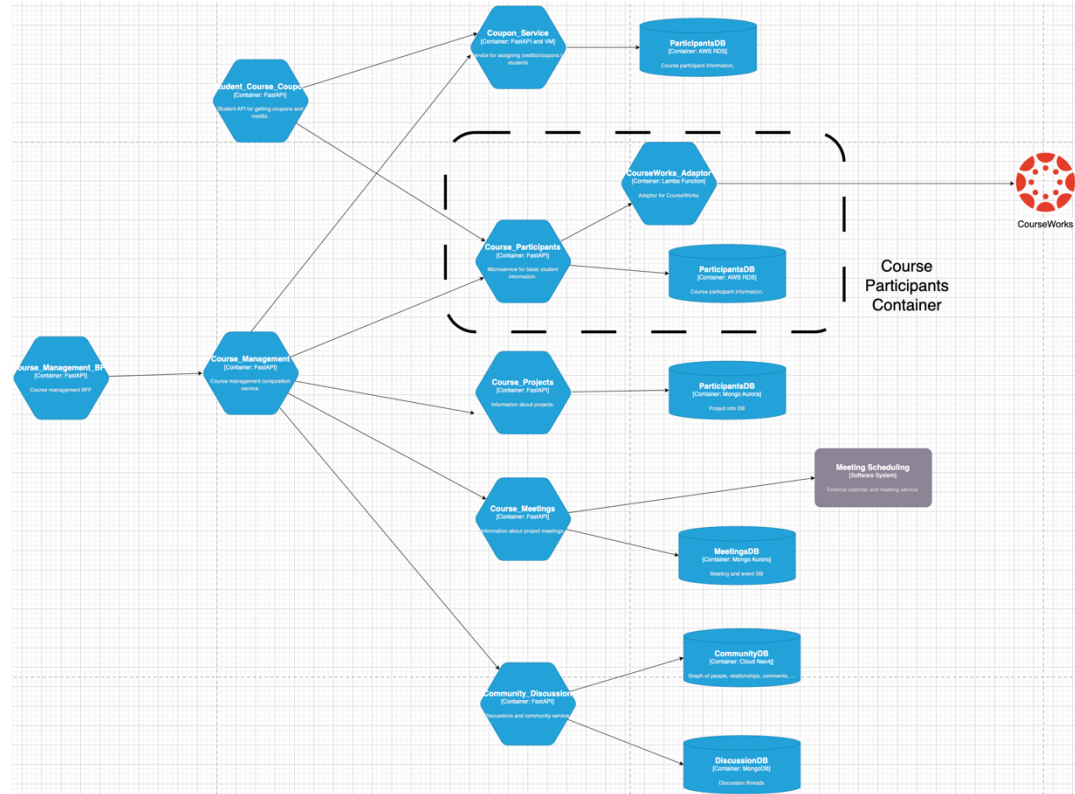
Notes to DFF

- Explain how this starts to realize the SOLID Principles
- Explain unit testing
- Just use placeholders for now in your microservices.
- These are placeholders for
 - Reuse
 - Flexibility, e.g. changing database engines and schema
 - Dependency injection
- Explain role of environment variables for local testing and deployment.

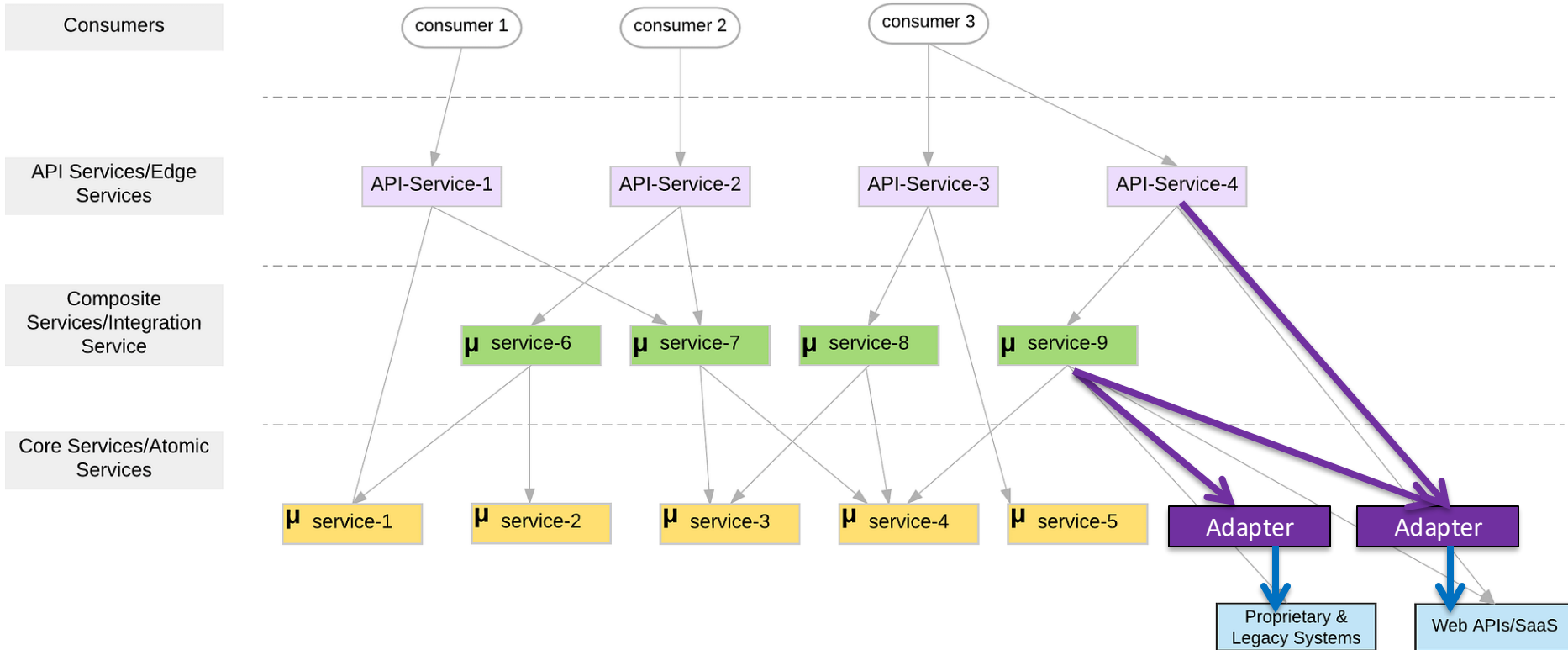
My First Steps

(Partial) Logical View – Interactive System

- Some of the distinctions are arbitrary or in the “eye of the beholder.”
 - System vs Container
 - Container vs Component
 - etc.
- A container is not synonymous with a “Docker container.”
- I have been in SW for 40 years and we still cannot define terms like “component.”
- Just use common sense.
- In general, containers in the cloud are *microservices*.”



Our First Design Patterns: Adapter, Anti-Corruption



Adapter and Anti-Corruption

Adapter Microservice

You are building an application that is following a Microservices architecture. The team is not operating in a complete greenfield - there are existing sources of functionality or data that must be reused in order to complete the application on time and within budget. However,

This existing functionality cannot be migrated to a microservice architecture. We need a way to be able to take advantage of this functionality without abandoning the microservices approach.

There are many different types of existing functionality that may require you to reuse existing code. It may be that there are specific enterprise data sources that can only be accessed through an existing API. Or it may be that you want to take advantage of a cloud-based (SaaS) API from a cloud provider that provides some or most of the functionality that you would otherwise have to build.

Therefore,

Wrap the functionality in an adapter. The adapter will (like the adapter pattern from [Gamma](#)) convert from the existing interface to a new interface consistent with microservice principles (e.g. REST or messaging).

Perhaps the hardest aspect to determining if you need to use an Adapter Microservice, particularly if the legacy service or SaaS service already implements a RESTful interface, is whether or not to build an Adapter Microservice or simply use the existing REST API directly from your SPA or Mobile Application clients or your BFF's. There are several reasons why you may want to build an Adapter Microservice of one type or another in order to improve the maintainability of your code.

<https://kgb1001001.github.io/cloudadoptionpatterns/>

Anti-corruption Layer pattern

Azure Azure Logic Apps

Implement a façade or adapter layer between different subsystems that don't share the same semantics. This layer translates requests that one subsystem makes to the other subsystem. Use this pattern to ensure that an application's design is not limited by dependencies on outside subsystems. This pattern was first described by Eric Evans in *Domain-Driven Design*.

Context and problem

Most applications rely on other systems for some data or functionality. For example, when a legacy application is migrated to a modern system, it may still need existing legacy resources. New features must be able to call the legacy system. This is especially true of gradual migrations, where different features of a larger application are moved to a modern system over time.

Often these legacy systems suffer from quality issues such as convoluted data schemas or obsolete APIs. The features and technologies used in legacy systems can vary widely from more modern systems. To interoperate with the legacy system, the new application may need to support outdated infrastructure, protocols, data models, APIs, or other features that you wouldn't otherwise put into a modern application.

Maintaining access between new and legacy systems can force the new system to adhere to at least some of the legacy system's APIs or other semantics. When these legacy features have quality issues, supporting them "corrupts" what might otherwise be a cleanly designed modern application.

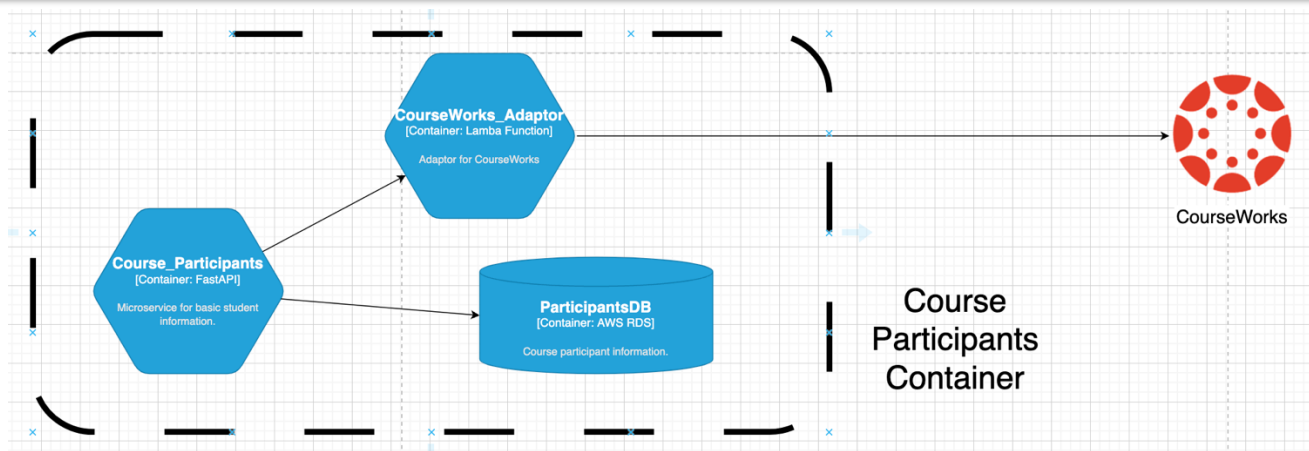
Similar issues can arise with any external system that your development team doesn't control, not just legacy systems.

Solution

Isolate the different subsystems by placing an anti-corruption layer between them. This layer translates communications between the two systems, allowing one system to remain unchanged while the other can avoid compromising its design and technological approach.

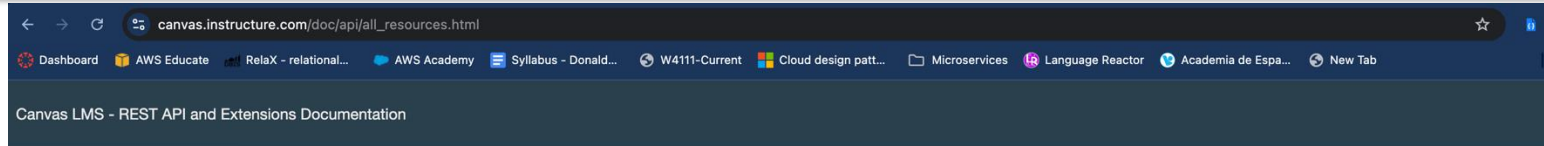
<https://learn.microsoft.com/en-us/azure/architecture/patterns/anti-corruption-layer>

CourseWorks Adaptor



- CourseWorks has an API. It is an HTTP based API. And is sort of “REST” like, but is kind of wonky. (https://canvas.instructure.com/doc/api/all_resources.html).
- I want to define a preferred, canonical model for courses, sections, participants for my solution.
- And map the model to CourseWorks, and have the option for changing.

CourseWorks/Canvas LMS API



Expand all

› Basics

› OAuth2

› Resources

› Outcomes

› Group Categories

› SIS

› External Tools

› Data Services

All API Resources

Access Tokens	Show an access token Create an access token Update an access token Delete an access token
Account Calendars	List available account calendars Get a single account calendar Update a calendar Update several calendars List all account calendars Count of all visible account calendars
Account Domain Lookups	Search account domains
Account Notifications	Index of active global notification for the user Show a global notification Close notification for user Create a global notification Update a global notification
Account Reports	List Available Reports Start a Report Index of Reports Status of a Report Delete a Report
Accounts	List accounts Get accounts that admins can manage Get accounts that users can create courses in List accounts for course admins Get a single account Settings List environment settings Permissions Get the sub-accounts of an account Get the Terms of Service Get help links Get the manually-created courses sub-account for the domain root account List active courses in an account Update an account Delete a user from the root account Restore a deleted user from a root account
Subaccounts	Create a new sub-account Delete a sub-account
Accounts (LTI)	Get account
Admins	Make an account admin Remove account admin List account admins List my admin roles
Analytics	Get department-level participation data Get department-level grade data Get department-level statistics Get department-level statistics, broken down by subaccount Get course-level participation data Get course-level assignment data Get course-level student summary data Get user-in-a-course-level participation data Get user-in-a-course-level assignment data Get user-in-a-course-level messaging data
Announcement External Feeds	List external feeds Create an external feed Delete an external feed

Applying Some Patterns/Principles

The first “real” microservice will initially demonstrate some concepts:

- 12 Factor App
 - Declare and isolate dependencies → requirements.txt
 - Store config in the environment → environment variables for API key, URLs
- SOLID Principles
 - Single Responsibility → a microservice just focused on CourseWorks.
 - Interface Segregation → the service only exposes the APIs necessary for scenario.
 - Liskov Substitution → the service will have an OpenAPI definition, and the implementation can be substituted over time.
- Design Patterns:
 - Adaptor
 - Anti-Corruption

These are concepts will hit later in your project. Your first microservice will be simpler.

But, my microservice exposes a REST API and I need to explain that first.

REST

REST (<https://restfulapi.net/>)

What is REST

- REST is acronym for **RE**presentational **State** Transfer. It is architectural style for **distributed hypermedia systems** and was first presented by Roy Fielding in 2000 in his famous [dissertation](#).
- Like any other architectural style, REST also does have its own [6 guiding constraints](#) which must be satisfied if an interface needs to be referred as **RESTful**. These principles are listed below.

Guiding Principles of REST

- **Client-server** – By separating the user interface concerns from the data storage concerns, we improve the portability of the user interface across multiple platforms and improve scalability by simplifying the server components.
- **Stateless** – Each request from client to server must contain all of the information necessary to understand the request, and cannot take advantage of any stored context on the server. Session state is therefore kept entirely on the client.
- **Cacheable** – Cache constraints require that the data within a response to a request be implicitly or explicitly labeled as cacheable or non-cacheable. If a response is cacheable, then a client cache is given the right to reuse that response data for later, equivalent requests.
- **Uniform interface** – By applying the software engineering principle of generality to the component interface, the overall system architecture is simplified and the visibility of interactions is improved. In order to obtain a uniform interface, multiple architectural constraints are needed to guide the behavior of components. REST is defined by four interface constraints: identification of resources; manipulation of resources through representations; self-descriptive messages; and, hypermedia as the engine of application state.
- **Layered system** – The layered system style allows an architecture to be composed of hierarchical layers by constraining component behavior such that each component cannot “see” beyond the immediate layer with which they are interacting.
- **Code on demand (optional)** – REST allows client functionality to be extended by downloading and executing code in the form of applets or scripts. This simplifies clients by reducing the number of features required to be pre-implemented.

Resources

Resources are an abstraction. The application maps to create things and actions.

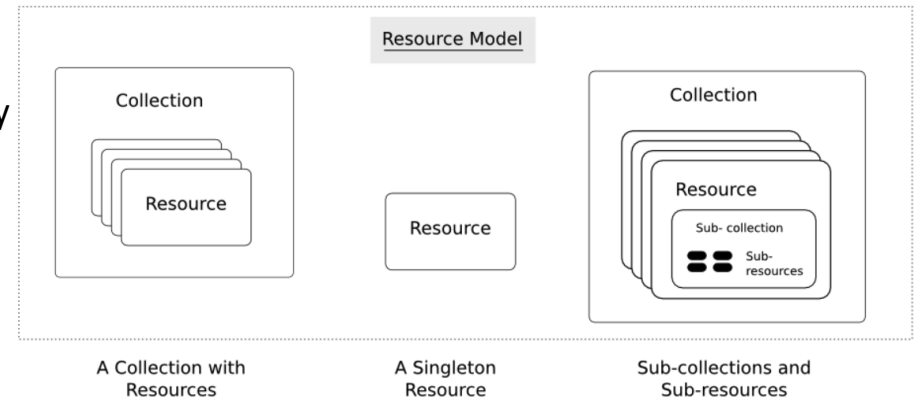
“A resource-oriented API is generally modeled as a resource hierarchy, where each node is either a *simple resource* or a *collection resource*. For convenience, they are often called a resource and a collection, respectively.

- A collection contains a list of resources of **the same type**. For example, a user has a collection of contacts.
- A resource has some state and zero or more sub-resources. Each sub-resource can be either a simple resource or a collection resource.

For example, Gmail API has a collection of users, each user has a collection of messages, a collection of threads, a collection of labels, a profile resource, and several setting resources.

While there is some conceptual alignment between storage systems and REST APIs, a service with a resource-oriented API is not necessarily a database, and has enormous flexibility in how it interprets resources and methods. For example, creating a calendar event (resource) may create additional events for attendees, send email invitations to attendees, reserve conference rooms, and update video conference schedules. (Emphasis added)

(<https://cloud.google.com/apis/design/resources#resources>)



<https://restful-api-design.readthedocs.io/en/latest/resources.html>

REST – Resource Oriented

- When writing applications, we are used to writing functions or methods:
 - `openAccount(last_name, first_name, tax_payer_id)`
 - `account.deposit(deposit_amount)`
 - `account.close()`

We can create and implement whatever functions we need.

- REST only allows four methods:
 - POST: Create a resource
 - GET: Retrieve a resource
 - PUT: Update a resource
 - DELETE: Delete a resource

“The key characteristic of a resource-oriented API is that it emphasizes resources (data model) over the methods performed on the resources (functionality). A typical resource-oriented API exposes a large number of resources with a small number of methods.”

(<https://cloud.google.com/apis/design/resources>)

That's it. That's all you get.

- A REST client needs no prior knowledge about how to interact with any particular application or server beyond a generic understanding of hypermedia.

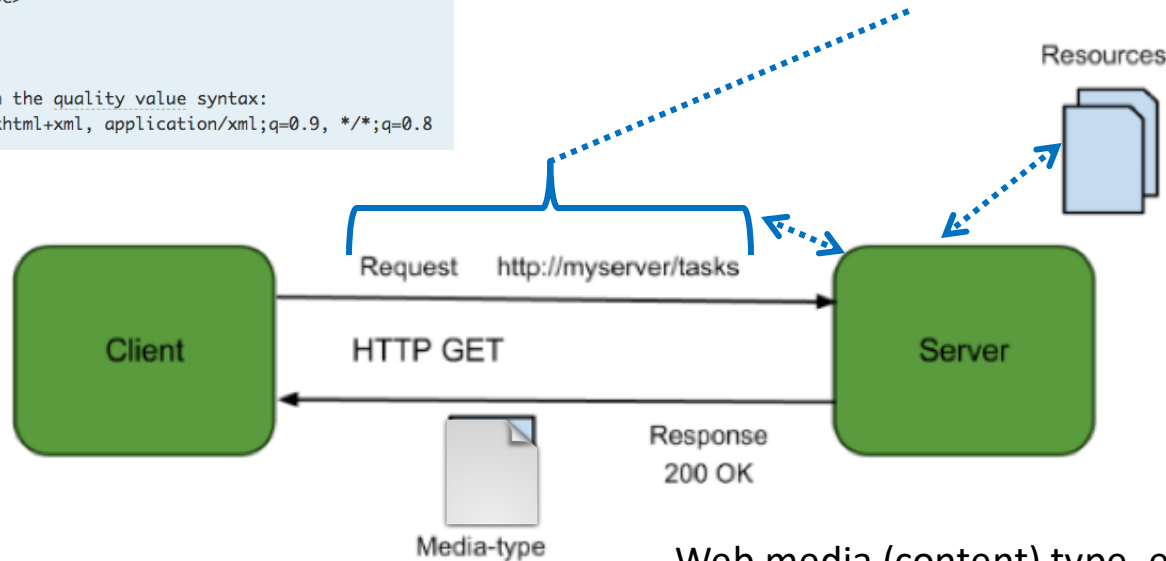
Resources, URLs, Content Types

Accept type in headers.

```
Accept: <MIME_type>/<MIME_subtype>
Accept: <MIME_type>/*
Accept: */*
```

```
// Multiple types, weighted with the quality value syntax:
Accept: text/html, application/xhtml+xml, application/xml;q=0.9, */*;q=0.8
```

- Relative URL identifies “resource” on the server.
- Server implementation maps abstract resource to tangible “thing,” file, DB row, ... and any application logic.



Client may be
Browser
Mobile device
Other REST Service
... ..

Web media (content) type, e.g.

- text/html
- application/json

REST Principles

- What about all of those principles?
 - Client/Server
 - Stateless
 - Cacheable
 - Uniform Interface
 - Layered System
 - Code on demand
- Some of these principles
 - Are simple and obvious.
 - Are subtle and have major implications.
 - Stateless can be baffling, but we will cover later.
- We will go into the principles in later lectures, ...
- We will also go into HATEOAS, GraphQL,

API First OpenAPI

API First

- “As the connective tissue linking ecosystems of technologies and organizations, APIs allow businesses to monetize data, forge profitable partnerships, and open new pathways for innovation and growth.” (McKinsey)
- “API-first, also called the API-first approach, prioritizes APIs at the beginning of the software development process, positioning APIs as the building blocks of software. API-first organizations develop APIs before writing other code, instead of treating them as afterthoughts. This lets teams construct applications with internal and external services that are delivered through APIs.”
(<https://www.netsolutions.com/hub/mach-architecture/api-first>)

Why API-First Approach?

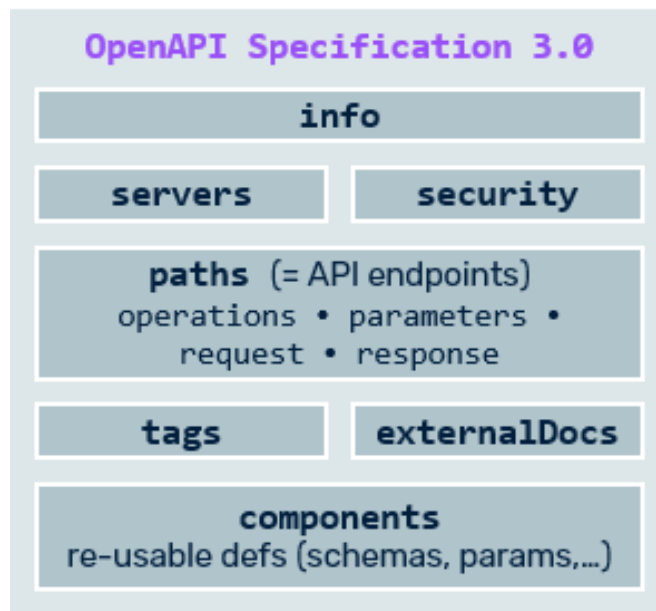
- By defining the APIs first, there is a modular separation between the services in the system. This increases the reusability of API endpoints across different parts of the system and to external entities as well (Interoperability).
- The API-first approach promotes collaboration as API documentation and contract definitions serve as a common language for stakeholders such as different teams in development (e.g. frontend and backend), V&V teams for test scenarios, etc.
- Creating the API contracts as part of the API-first design approach can be shared with frontend teams where the team can start building user interfaces parallelly with mocked data as per contract models, tested for the user experience. Also, codegen can be used to generate client mocks which can be directly used by client apps such as iOS/Android, etc.
- It gives the flexibility for tech-stack choices. Different services or applications in the system can be in different languages as long as they adhere to API contracts. It can help in identifying the best suitable tools/options in terms of cost and performance.
- With the API-First approach many concerns such as security measures, versioning, request traffic and data volume, and Integration with different services/applications within the system.

<https://medium.com/@rohitranjan.pandey/api-first-design-approach-swagger-a1423db48f76>



OpenAPI 3.0

- OpenAPI is a standard for documenting REST APIs.
- An HTTP request/response has several elements:
 - Protocol
 - Server address and port
 - Path
 - Query parameters
 - Fragment
 - Headers, Cookies
 - Body content and content type
 - Method
- OpenAPI and web application frameworks introduce:
 - Path parameters
 - Models to represent bodies/content
 - Security information
 - Descriptive metadata



serialized in either
JSON or YAML

HTTP, OAuth2, JWT³

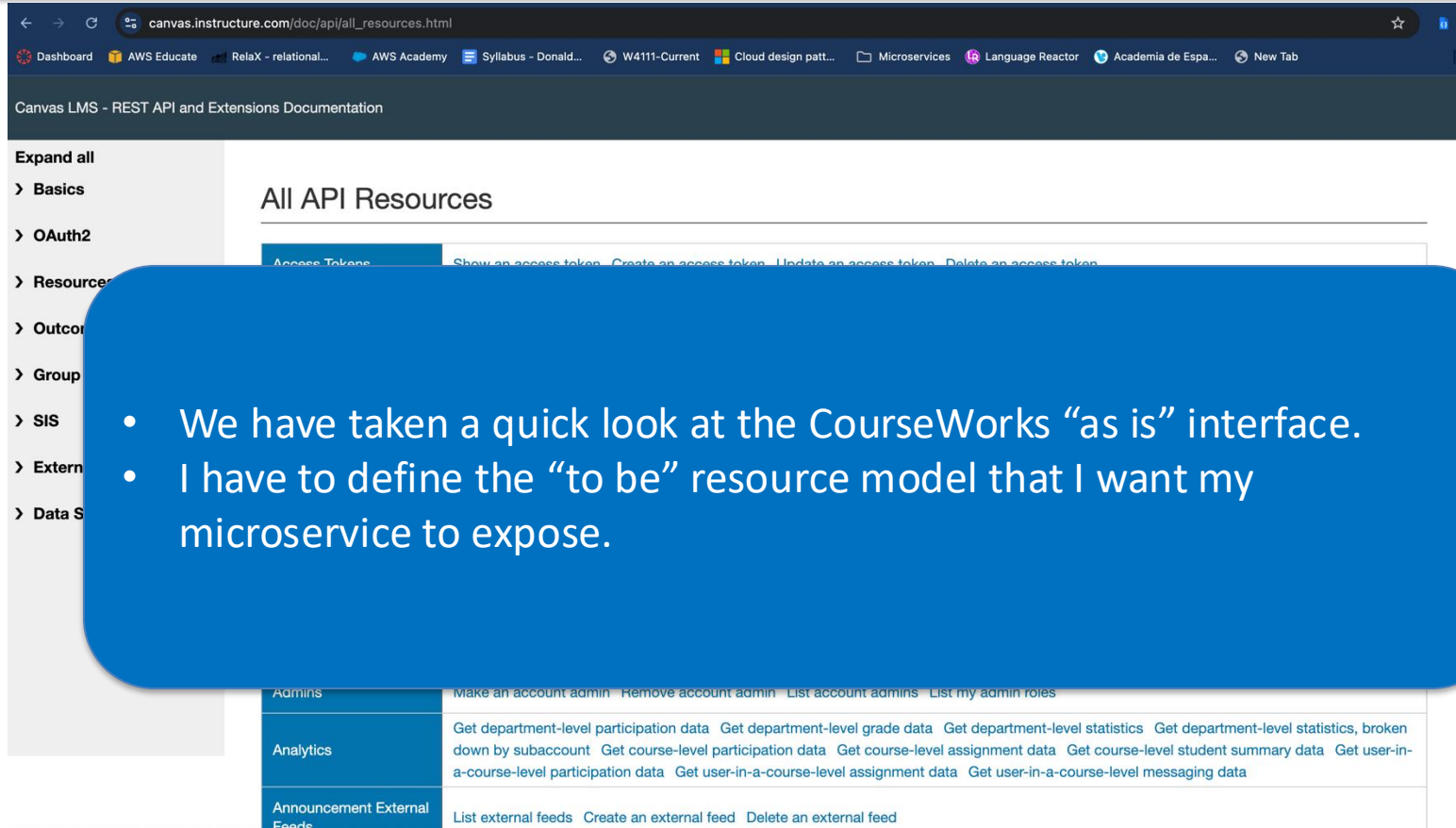
■ synchronous calls
■ call backs

URL Anatomy

https://example.com:80/blog?search=test&sort_by=created_at#header

↓ ↓ ↓ ↓ ↓ ↓
Protocol Domain Port Path Query Parameters Fragment/Anchor

CourseWorks/Canvas LMS API



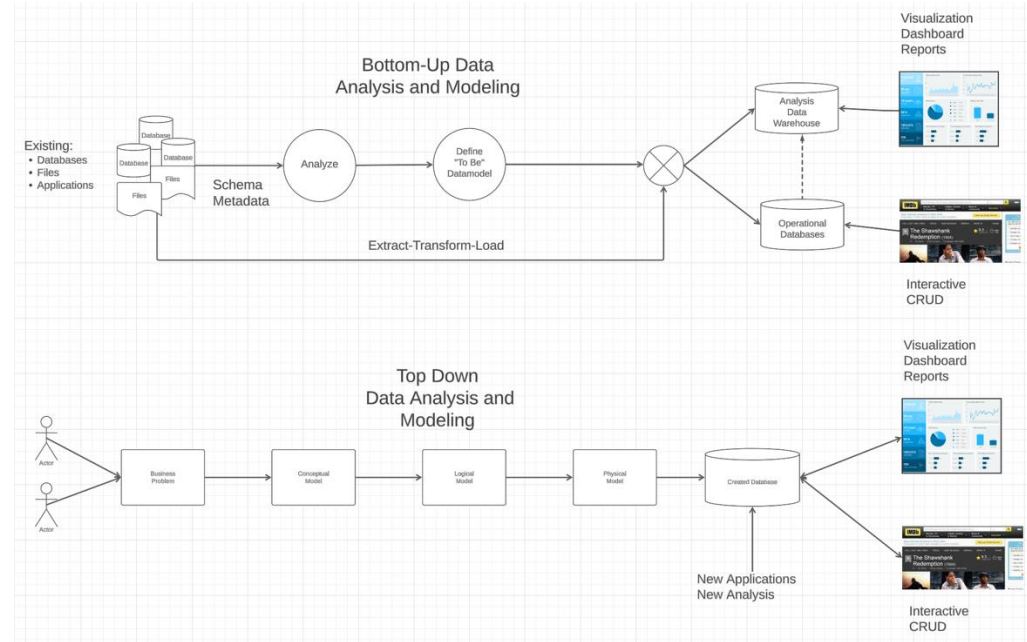
The screenshot shows the Canvas LMS REST API documentation page. The browser address bar displays 'canvas.instructure.com/doc/api/all_resources.html'. The page title is 'Canvas LMS - REST API and Extensions Documentation'. A sidebar on the left lists various API categories: Basics, OAuth2, Resource, Outcomes, Groups, SIS, External, and Data Services. The main content area is titled 'All API Resources' and lists several API endpoints with their corresponding actions. A large blue rounded rectangle is overlaid on the page, containing the following text:

- We have taken a quick look at the CourseWorks “as is” interface.
- I have to define the “to be” resource model that I want my microservice to expose.

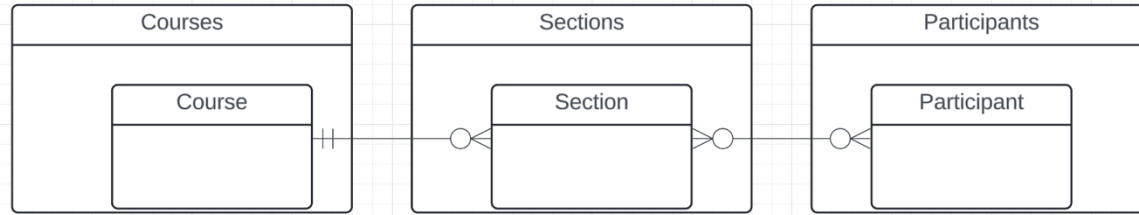
Below the blue overlay, the documentation table is partially visible, showing categories like Admins, Analytics, and Announcement External Feeds with their respective API endpoints.

Top-Down, Bottom-Up, Meet-in-the-Middle

- In my DB class, I cover:
 - Top-down data modeling
 - Bottom-up data modeling
- An adaptor implements meet-in-the-middle and converts
 - What exists to
 - What my application wants
- REST talks about “resources.”
- API design is similar to data modeling. Many concepts have analogs in REST. API First is like top-down modeling in a DB.



Resources and APIs



- Base resources, paths and methods:
 - /courses: GET, POST
 - /courses/<id>: GET, PUT, DELETE
 - /sections: GET, POST
 - /sections/<id>: GET, PUT, DELETE
 - /participants: GET, POST
 - /participants/<id>: GET, PUT, DELETE
- There are relative, navigation paths:
 - /courses/<id>/sections
 - /participants/<id>/sections
 - etc.
- GET on resources that are collections may also have query parameters.
- There are two approaches to defining API
 - Start with OpenAPI document and produce an implementation template.
 - Start with annotated code and generate API document.
- In either approach, I start with *models*.
- Also,
 - I lack the security permission to update CourseWorks.
 - I can choose to not surface the methods or raise and exception.

API First Starting with OpenAPI

The screenshot displays the API Hub Design interface for a project named "Columbia-Classes". The main area shows an OpenAPI 3.0.0 specification for a "PersonInfo" API. The specification includes a title, description, servers, and several endpoints: GET /persons, POST /persons, GET /persons/{uni}, PUT /persons/{uni}, and DELETE /persons/{uni}. The GET /persons endpoint is highlighted, showing its summary, operationId, parameters, and responses.

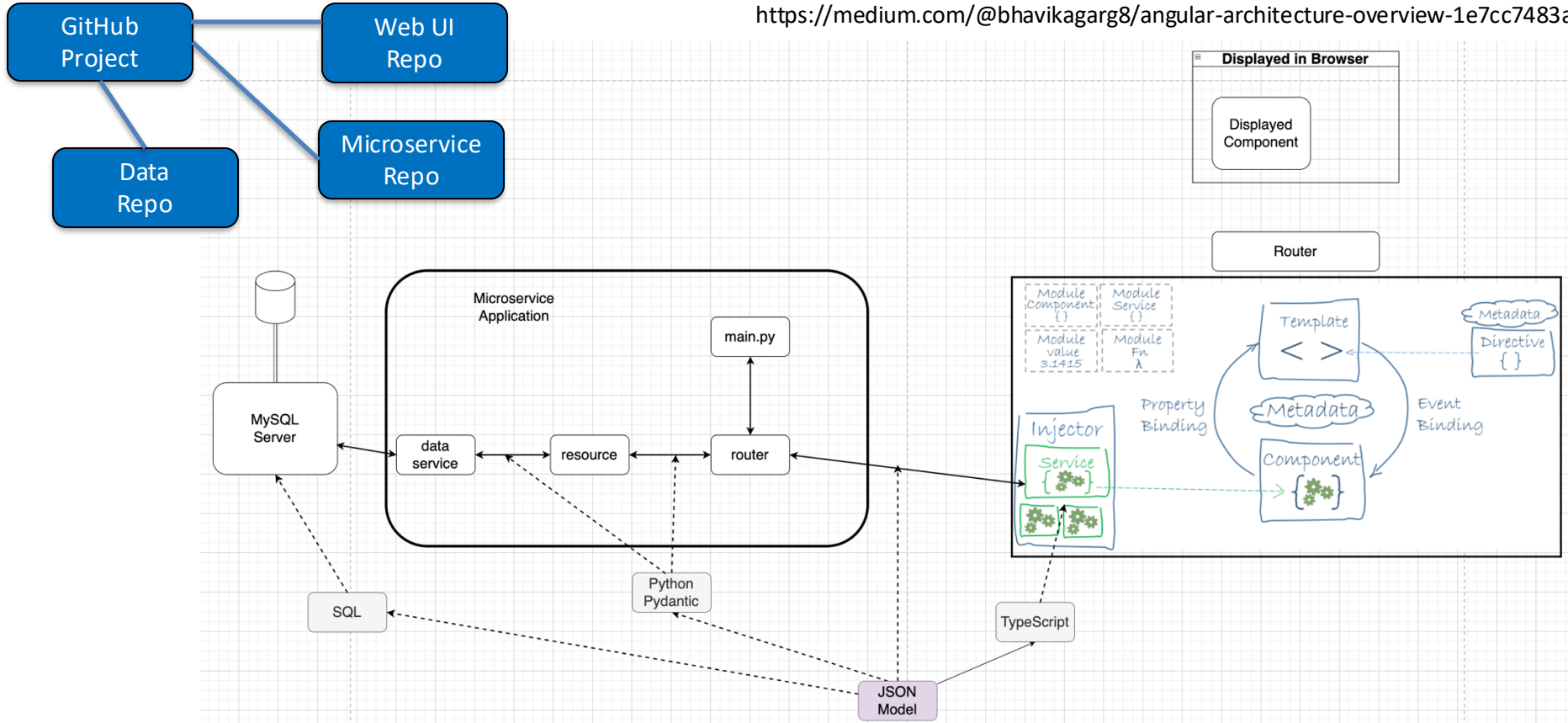
```
1 openapi: 3.0.0
2 info:
3   version: '1.0'
4   title: PersonInfo
5   description: Simple information about people associated with courses
6 servers:
7   - Added by API Auto Mocking Plugin
8     - description: SwaggerHub API Auto Mocking
9     url: https://virtserver.swaggerhub.com/Columbia-Classes/PersonInfo/1.0
10
11 paths:
12   /persons:
13     get:
14       summary: Search for persons by query parameters with pagination
15       operationId: persons
16       parameters:
17         - name: last_name
18           in: query
19           required: false
20           schema:
21             type: string
22         - name: first_name
23           in: query
24           required: false
25           schema:
26             type: string
27         - name: email
28           in: query
29           required: false
30           schema:
31             type: string
32         - name: page
33           in: query
34           required: false
35           schema:
36             type: integer
37             minimum: 1
38             default: 1
39           description: The page number to retrieve.
40         - name: size
41           in: query
42           required: false
43           schema:
44             type: integer
45             minimum: 1
46             default: 10
47           description: The number of records per page.
48       responses:
49         '200':
50           description: List of persons that match the query
51           headers:
52             Location:
53               description: The URL to access the next page of results or the queried resource.
```

The right sidebar shows the "PersonInfo" API details, including the title, version (1.0), and description. Below this, the "Servers" section shows the default server URL. The "default" section lists the API endpoints with their methods and descriptions. The "Schemas" section shows the "personInfo" schema.

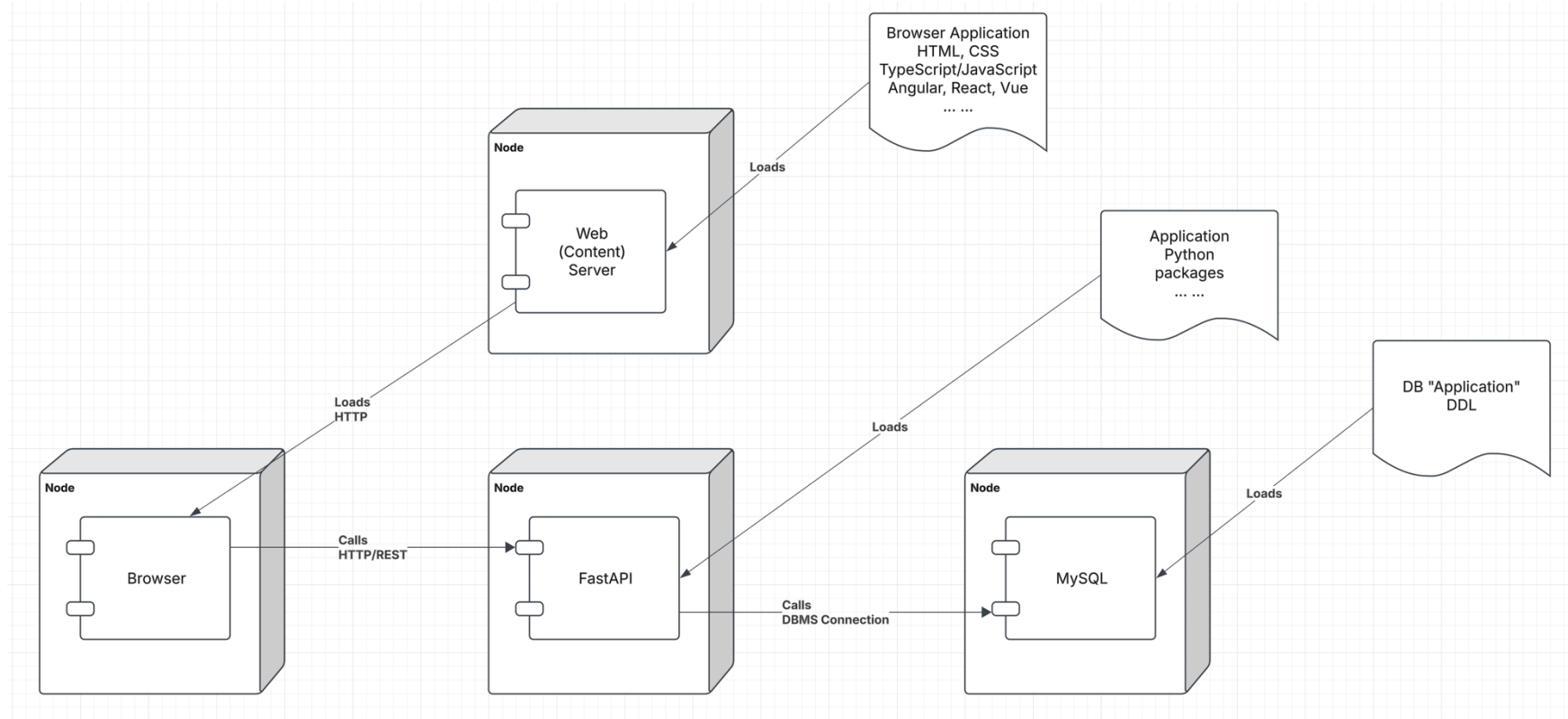
Full Stack Web Application

Full Stack Application Structure

<https://medium.com/@bhavikagarg8/angular-architecture-overview-1e7cc7483a0>



Full-Stack Web Application

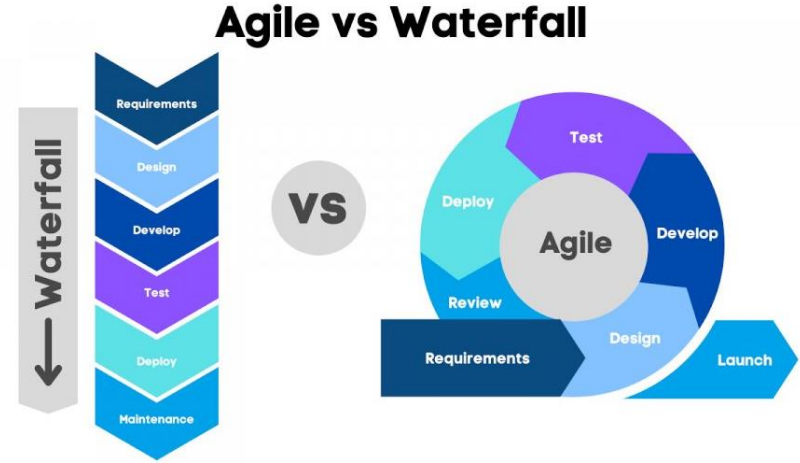


Agile Development

Agile Development

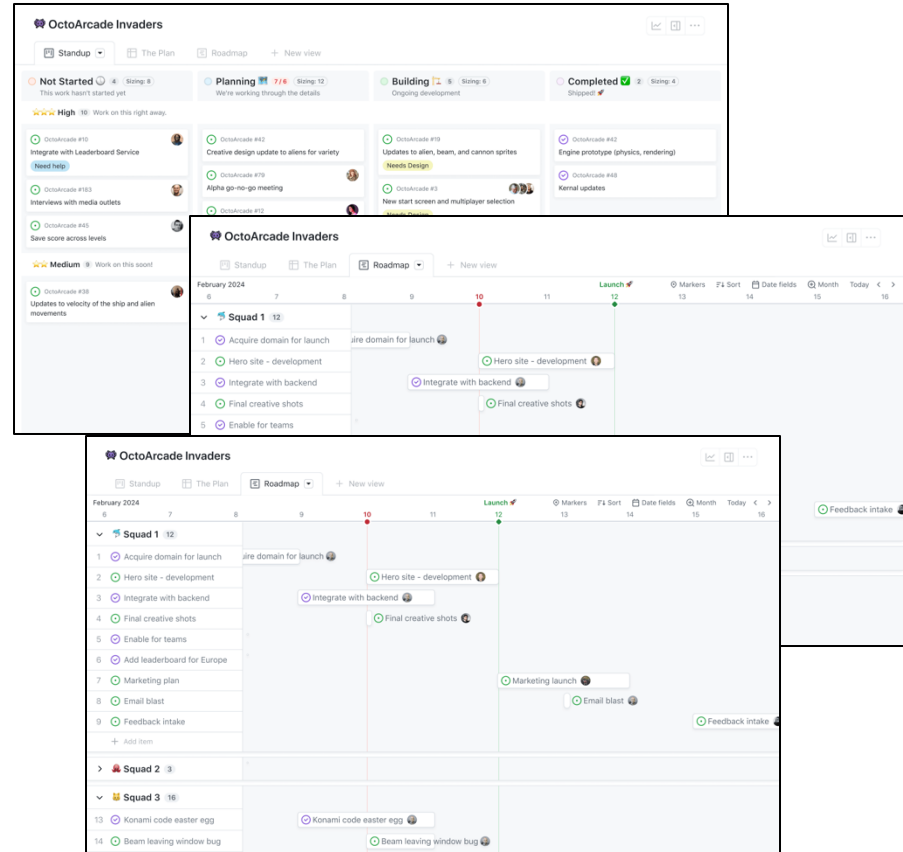
<https://www.linkedin.com/pulse/waterfall-vs-agile-which-better-you-why-datacademy-cloud>

- SW projects traditionally/ previously followed a waterfall model:
 - 12-18 months between releases.
 - Proceed through phases one at a time.
- Modern development follows a form of *agile development*.
 - A sequence of short sprints, with a working system at the end of each sprint.
 - Continuous, iterative, incremental refinement and improvement.
- In this course, project teams will execute in two-week sprints.
 - Start sprint by defining sprint objectives and user stories.
 - End of sprint status report and demo, which we will review.



Agile and GitHub Projects

- Teams will use GitHub Projects to track and manage their work.
 - Ideas
 - Issues
 - Backlog
 - Sprint
 -
- The “project” will reference subprojects and code repos.
- We will also get some experience with pull requests and CI/CD.



First Sprint

My First Sprint

Logical Model

1. One browser application served by web server.
2. There microservices implemented using FastAPI
 - a. Student-Profile managed basic info about students.
 - b. Projects-Teams manages student teams, membership and project titles.
 - c. Student-Coupon managed allocation of cloud credits to students.
3. Relational DBMS: One database server, holding 3 databases with one per microservice.

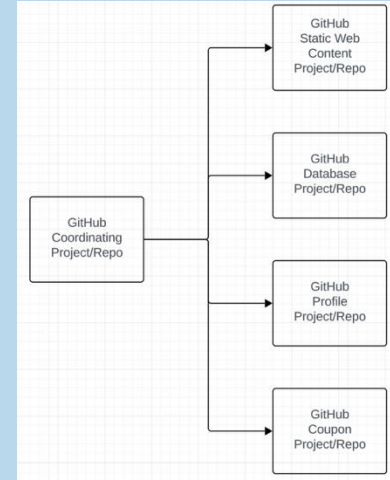
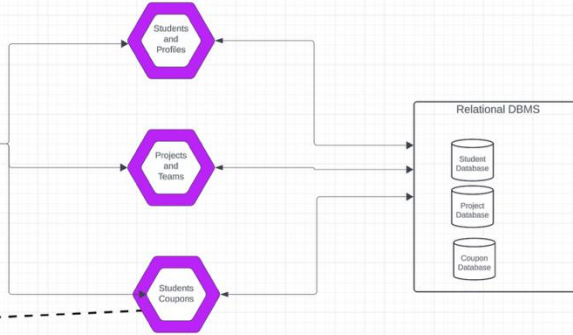


It is OK if this is just the default OpenAPI documents for now as long as you have a simple starter, "Hello World" UI application.

Purple wrapper indicates Hexagonal Architecture, which we will discuss in the semester.



This can be a placeholder as long as you have a simple "Hello World" application,

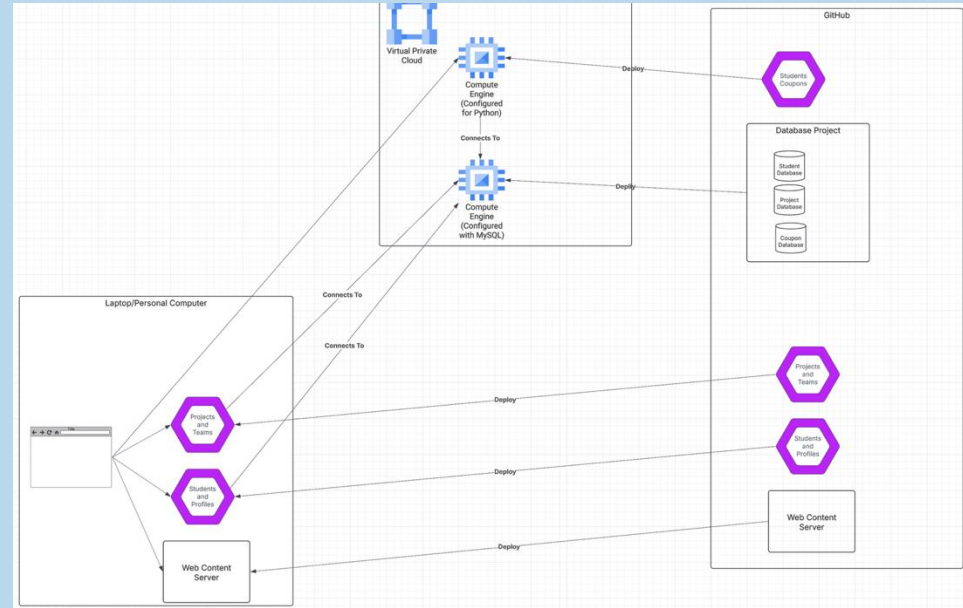
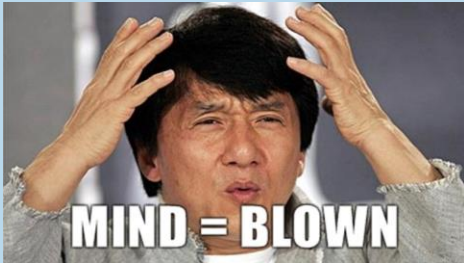


- Three microservices implementing path signatures but returning Not Implemented. At least one uses Swagger/SwaggerHub to define OpenAPI interface definition.
- A template/starter browser/UI GitHub repo/project containing static content, Angular,
- One GitHub repo/project per microservice.
- A GitHub repo/project for the database/data subsystem component. One database/microservice.
- Next slide will discuss deployment and remainder of the project.

First Sprint Deployment Architecture

Gain experience with IaaS

- Two VMs on Google Compute Engine in a VPC
 - 1 configured for/with Python
 - 1 configured for/with MySQL
- Deployments
 - 1 microservice on Python VM
 - Databases installed/created on DB VM
 - Two microservices on laptop/PC
 - Static content serving on PC/laptop



For now, focus on:

- Creating the VM and following instructions to install install SW.
- Creating starter/template GitHub projects and *local testing*.
- Following online tutorials, OHs, recitations, etc. for setting up GitHub,

Let's Talk about This

- Walk through the configuration of my VMs, including VPC firewall rules.
- Talk about SW installation and configuration steps
 - Web application system
 - Installing Python
 - git clone of the repository
 - Virtual environment and requirements.txt
 - Running application
 - Testing mysql connection
 - MySQL system
 - Installing MySQL was a bear. ChatGPT helped.
 - Need to configure remote access for mysql, create users, etc.
- Do not worry about connecting from the desktop microservices to the MySQL VM.
- You will submit a status report, from a template, in two weeks.